

Trabajo práctico Nro 1

Filsinger Gonzalo, Perea Ignacio, Leon Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En este informe se abordará el desarrollo del trabajo práctico Nro 2, en el cual experimentaremos con las interrupciones externas del microcontrolador STM32F103C8T6, utilizando la placa BluePill. Se desarrollaran los primeros dos ejercicios del trabajo práctico.

Index Terms—BluePill, GPIO, LED, Memoria, Interrupciones, Botones.

I. INTRODUCCIÓN

EN este informe solamente se abordarán los primeros dos ejercicios del trabajo práctico, en el primero utilizaremos dos botones uno con mas prioridad que el otro para interrumpir el parpadeo de un led, y en el segundo ejercicio haremos uso del ADC para medir la tensión de salida de un potenciómetro (Hasta 3.3V), y temperatura de un sensor interno, según un boton que cambia entre la lectura del potenciómetro y la lectura del sensor de temperatura.

II. EJERCICIO 1: INTERRUPTIONES CON BOTONES DE BAJA Y ALTA PRIORIDAD

Para este programa se usará el código del TP1 como base, el cual se modificará para hacer interrupciones al led del TP1 con botones de baja y alta prioridad. Para conseguir esto se agregaron las siguientes definiciones en el archivo main.c:

- Se definió la dirección base **AFIO_BASE**
- Se definió la dirección base **EXTI_BASE**
- Se definió la dirección base **NVIC_BASE**
- Se definió el registro **AFIO_EXTICR1**
- Se definió el registro **EXTI_IMR**
- Se definió el registro **EXTI_FTSR**
- Se definió el registro **EXTI_PR**
- Se definió el registro **NVIC_ISER0**
- Se definió el registro **NVIC_IPR**
- Se definieron las mascaras **GPIOA6** y **GPIOA7**

```
#include <stdint.h>

// --- Direcciones Base ---
#define RCC_BASE    0x40021000
#define GPIOC_BASE  0x40011000
#define GPIOA_BASE  0x40010800
#define AFIO_BASE   0x40010000
#define EXTI_BASE   0x40010400
#define NVIC_BASE   0xE000E100

// --- Registros de Periféricos ---
#define RCC_APB2ENR  *(volatile uint32_t *) (RCC_BASE + 0x18)
#define GPIOC_CRH    *(volatile uint32_t *) (GPIOC_BASE + 0x04)
#define GPIOA_CRL    *(volatile uint32_t *) (GPIOA_BASE + 0x00)
#define GPIOC_ODR    *(volatile uint32_t *) (GPIOC_BASE + 0x0C)
#define GPIOA_ODR    *(volatile uint32_t *) (GPIOA_BASE + 0x0C)

#define AFIO_EXTICR1 *(volatile uint32_t *) (AFIO_BASE + 0x08)
#define EXTI_IMR     *(volatile uint32_t *) (EXTI_BASE + 0x00)
#define EXTI_FTSR    *(volatile uint32_t *) (EXTI_BASE + 0x0C)
#define EXTI_PR      *(volatile uint32_t *) (EXTI_BASE + 0x14)
#define NVIC_ISER0   *(volatile uint32_t *) (NVIC_BASE + 0x00)
#define NVIC_IPR     ((volatile uint8_t *) (NVIC_BASE + 0x400))

// --- Campos de Bits ---
#define RCC_AFIOEN   (1 << 0)
#define RCC_IOPAEN   (1 << 2)
#define RCC_IOPCEN   (1 << 4)
#define GPIOC13      (1UL << 13)
#define GPIOA7       (1UL << 7) // LED Normalidad
#define GPIOA6       (1UL << 6) // LED Botón Baja Prioridad
#define GPIOA5       (1UL << 5) // LED Botón Alta Prioridad
```

Definiciones de los registros en main.c

Aquí se configuran los registros necesarios para poder realizar las interrupciones con los botones. El registro **AFIO_EXTICR1** se utiliza para configurar el pin del botón como fuente de interrupción externa; en este caso usaremos dos, el A1 y el A2. El registro **EXTI_IMR** se utiliza para habilitar las interrupciones de los pines configurados. El registro **EXTI_FTSR** se utiliza para configurar la interrupción por flanco de bajada. El registro **NVIC_ISER0** se utiliza para habilitar las interrupciones en el NVIC, y el registro **NVIC_IPR** se utiliza para configurar la prioridad de las interrupciones.

A continuación agregamos las funciones de interrupción para los botones, las cuales se encargan de encender y apagar el led correspondiente al botón presionado. Para el botón de

baja prioridad (A1) se encenderá el led del pin A6, y para el botón de alta prioridad (A2) se encenderá el led del pin A7.

```
// Botón en PA1 (EXTI1) - PRIORIDAD BAJA -> Titila PA6 5 veces
void EXTI1_IRQHandler(void)
{
    if (EXTI_PR & (1 << 1))
    {
        // Limpiamos el flag primero para que futuras pulsaciones se registren correctamente
        EXTI_PR |= (1 << 1);

        // Encendemos y apagamos 5 veces
        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA6;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA6;
    }
}
```

Agregado de funciones para interrupciones

```
// Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila PA5 5 veces
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        // Limpiamos el flag
        EXTI_PR |= (1 << 2);

        // Encendemos y apagamos 5 veces
        for (int i = 0; i < 10; i++)
        {
            GPIOA_ODR ^= GPIOA5;
            delay_ms_bloqueante(200);
        }

        GPIOA_ODR &= ~GPIOA5;
    }
}
```

Agregado de funciones para interrupciones

Ahora pasamos a modificar la función **main**

```
void main(void)
{
    // 1. RCC (Relojes)
    RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN | RCC_IOPCEN;

    // 2. GPIO
    // PC13 (LED placa) como salida
    GPIOC_CRH &= 0xFF0FFFFF;
    GPIOC_CRH |= 0x00200000;

    // Configurar PA7, PA6 y PA5 como SALIDAS push-pull (2)
    // Configurar PA2 y PA1 como ENTRADAS pull-up/down (8)
    GPIOA_CRL &= 0x000FF00F;
    GPIOA_CRL |= 0x22200880;

    // Activamos resistencias Pull-up internas en PA1 y PA2
    GPIOA_ODR |= (1 << 1) | (1 << 2);

    // 3. AFIO (Multiplexor)
    AFIO_EXTICR1 &= ~(0x00000FF0);

    // 4. EXTI
    EXTI_IMR |= (1 << 1) | (1 << 2);
    EXTI_FTSR |= (1 << 1) | (1 << 2);

    // 5. NVIC PRIORIDADES (Ajustadas para permitir que SysTick no se congele)
    NVIC_IPR[7] = 0x20; // Prioridad BAJA para Botón PA1 (IRQ 7)
    NVIC_IPR[8] = 0x10; // Prioridad ALTA para Botón PA2 (IRQ 8)
    // Nota: SysTick mantiene su prioridad por defecto que es 0x00 (Máxima)

    // 6. NVIC ENABLE
    NVIC_ISER0 |= (1 << 7) | (1 << 8);

    systick_init_ms();

    while (1)
    {
        // --- Tarea de Normalidad (Background Task) ---
        GPIOA_ODR ^= GPIOA7;
        GPIOC_ODR ^= GPIOC13;
        delay_ms_bloqueante(500);
    }
}
```

Configuración de los pines y las interrupciones en main

Primero habilitamos los clocks del GPIO y AFIO, luego configuramos los pines A5, A6 y A7 como salida, y los A1 y A2 como entrada con pull-up. Luego asociamos el pin GPIO a la línea EXTI y pasamos a configurar las interrupciones como flancos descendentes, por ultimo configuramos el NVIC con las prioridades correspondientes.

II-A. Imagenes del Funcionamiento

III. EJERCICIO 2: USO DE ADC

Ahora se busca controlar mediante el ADC unos leds que registran el voltaje de salida de un potenciómetro, o la opción 2, leer la temperatura del sensor interno del microcontrolador, como el ADC se configura en el PA1, usaremos el PA2 para el botón. Para esto se agregaron las siguientes definiciones en el archivo main.c:

```
// --- Registros del ADC1 ---
#define ADC1_BASE 0x40012400
#define ADC1_SR *(volatile uint32_t *) (ADC1_BASE + 0x00)
#define ADC1_CR2 *(volatile uint32_t *) (ADC1_BASE + 0x08)
#define ADC1_SMPR1 *(volatile uint32_t *) (ADC1_BASE + 0x0C)
#define ADC1_SMPR2 *(volatile uint32_t *) (ADC1_BASE + 0x10)
#define ADC1_SQR3 *(volatile uint32_t *) (ADC1_BASE + 0x34)
#define ADC1_DR *(volatile uint32_t *) (ADC1_BASE + 0x4C)

// --- Campos de Bits ---
#define RCC_AFIEN (1 << 0)
#define RCC_IOPAEN (1 << 2)
#define RCC_IOPCEN (1 << 4)
#define RCC_ADC1EN (1 << 9)

#define GPIOC13 (1UL << 13)

#define GPIOA3 (1UL << 3) // Nuevo LED 1
#define GPIOA4 (1UL << 4) // Nuevo LED 2
#define GPIOA5 (1UL << 5) // Nuevo LED 3
#define GPIOA6 (1UL << 6) // Nuevo LED 4

#define ADC_CR2_ADON (1 << 0)
#define ADC_CR2_CAL (1 << 2)
#define ADC_CR2_EXTSEL (7 << 17)
#define ADC_CR2_EXTTRIG (1 << 20)
#define ADC_CR2_SWSTART (1 << 22)
#define ADC_CR2_TSVREFE (1 << 23)
#define ADC_SR_EOC (1 << 1)
```

Definiciones de los registros para interrupciones

- Se definió el registro **ADC1_BASE** con la dirección base del ADC, el cual se utiliza para configurar y controlar el funcionamiento del ADC.
- Se definió el registro **ADC1_SR** el cual se utiliza para verificar el estado del ADC, como por ejemplo si la conversión ha terminado o si hay algún error.
- Se definió el registro **ADC_CR2** el cual se utiliza para configurar el ADC, como por ejemplo habilitar el ADC, configurar el modo de conversión, etc.
- Se definió el registro **ADC_SMPR1** el cual se utiliza para configurar el tiempo de muestreo del ADC, lo cual es importante para obtener una conversión precisa.
- Se definió el registro **ADC_SMPR2** el cual se utiliza para configurar el tiempo de muestreo del ADC para los canales 0 a 9.
- Se definió el registro **ADC_SQR3** el cual se utiliza para configurar el canal que se va a convertir.
- Se definió el registro **ADC_DR** el cual se utiliza para leer el resultado de la conversión del ADC.

Luego se configuró que al presionar el botón se cambie el canal del ADC entre el potenciómetro y el sensor de temperatura, para esto se configuró una interrupción externa en el pin PA2, la cual se activa al detectar un flanco de subida.

En la rutina de servicio de interrupción se cambia el canal del ADC entre el potenciómetro y el sensor de temperatura.

```
volatile uint8_t modo_adc = 0;

// --- ISR del Botón (Pin PA2) ---
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        EXTI_PR = (1 << 2);
        modo_adc = !modo_adc;
    }
}
```

Diagrama de conexiones para el ejercicio 2

Ahora lo nuevo que se implementa es el conversor ADC, el cual se inicializa de la siguiente forma:

```
// Inicializar ADC
ADC1_CR2 |= ADC_CR2_ADON;
delay_bucle(5000);
ADC1_CR2 |= ADC_CR2_ADON;

ADC1_CR2 |= ADC_CR2_CAL;
while(ADC1_CR2 & ADC_CR2_CAL);

ADC1_CR2 |= ADC_CR2_TSVREFE;

ADC1_CR2 |= ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;

ADC1_SMPR1 |= (0x7 << 18);
ADC1_SMPR2 |= (0x7 << 3);
```

Inicialización del ADC

Ahora se programa la función main en la cual se programa de que PIN se leerá el valor del ADC, y se muestra el resultado en los leds.

```

while (1)
{
    GPIOC_ODR ^= GPIOC13;

    if (modo_adc == 0) {
        ADC1_SQR3 = 16;
    } else {
        ADC1_SQR3 = 1;
    }

    // Iniciar conversión
    ADC1_CR2 |= ADC_CR2_SWSTART;

    // Esperar EOC (End Of Conversion)
    while ((ADC1_SR & ADC_SR_EOC) == 0);

    valor_adc = ADC1_DR;

    // Apagamos los 4 pines de la barra
    GPIOA_ODR &= ~(GPIOA6 | GPIOA5 | GPIOA4 | GPIOA3);

    // Encendido progresivo acumulativo
    if (valor_adc > 819) GPIOA_ODR |= GPIOA3; // Primer led
    if (valor_adc > 1638) GPIOA_ODR |= GPIOA4; // Segundo led
    if (valor_adc > 2457) GPIOA_ODR |= GPIOA5; // Tercer led
    if (valor_adc > 3276) GPIOA_ODR |= GPIOA6; // Cuarto led

    delay_bucle(30000);
}

```

Función main para el ejercicio 2

- Se configuró el registro **ADC1_SQR3** es el registro de Secuencia Regular 3. Aquí le estamos diciendo al microcontrolador qué canal físico debe conectar al núcleo del ADC en este momento. Si es 1 lee el PA1 y si es 16 lee el sensor de temperatura interno.
- Se configuró el registro **ADC1_CR2** el cual es Registro de Control 2 del ADC y es donde se configura su comportamiento. Al operar este registro con **ADC_CR2_SWSTART** (SoftWare Start) estamos haciendo que lea voltaje en ese momento.
- Polling es el término que se utiliza cuando se configura una espera bloqueante para que se pueda realizar la conversión. Dentro del while se encuentra **ADC1_SR & ADC_SR_EOC**, el cual opera El registro de estado del ADC con el bit de fin de conversión (EOC). Mientras se está realizando la conversión el bit EOC se mantiene en 0, y cuando la conversión termina el bit EOC se pone en 1.
- **Encendido progresivo de leds** en esta cadena de if se va encendiendo un led cada vez que el valor del ADC supera un umbral específico. Por ejemplo, si el valor del ADC es mayor a 819, se enciende el primer led, si es mayor a 1638 se enciende el segundo led, y así sucesivamente.

IV. CONCLUSIÓN

Se logró poner en funcionamiento la BluePill, y se logró hacer andar un led usando sus direcciones GPIO. Para esto

se tuvo que usar los comandos para la placa clon, además de modificar el Makefile para añadir dicho comando y así que funcione de manera correcta. Lo más difícil de este trabajo fue comenzar a pensar el código en base a los registros, ya que es un nivel de abstracción al que no estábamos acostumbrados, pero una vez que logramos comprender el funcionamiento del código logramos comprender mejor como funciona un microcontrolador a nivel de hardware, nos resultó particularmente interesante como un programa de alto nivel pasa a ser encender o apagar determinados bits de ciertos registros.

V. CODIGOS

V-A. main.c(Ejercicio 1)

```

1 #include <stdint.h>
2
3 // --- Direcciones Base ---
4 #define RCC_BASE 0x40021000
5 #define GPIOC_BASE 0x40011000
6 #define GPIOA_BASE 0x40010800
7 #define AFIO_BASE 0x40010000
8 #define EXTI_BASE 0x40010400
9 #define NVIC_BASE 0xE000E100
10
11 // --- Registros de Periféricos ---
12 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
13 + 0x18)
14 #define GPIOC_CRH *(volatile uint32_t *) (
15 GPIOC_BASE + 0x04)
16 #define GPIOA_CRL *(volatile uint32_t *) (
17 GPIOA_BASE + 0x00)
18 #define GPIOC_ODR *(volatile uint32_t *) (
19 GPIOC_BASE + 0x0C)
20 #define GPIOA_ODR *(volatile uint32_t *) (
21 GPIOA_BASE + 0x0C)
22
23 #define AFIO_EXTICR1 *(volatile uint32_t *) (
24 AFIO_BASE + 0x08)
25 #define EXTI_IMR *(volatile uint32_t *) (
26 EXTI_BASE + 0x00)
27 #define EXTI_FTSR *(volatile uint32_t *) (
28 EXTI_BASE + 0x0C)
29 #define EXTI_PR *(volatile uint32_t *) (
30 EXTI_BASE + 0x14)
31 #define NVIC_ISER0 *(volatile uint32_t *) (
32 NVIC_BASE + 0x00)
33
34 // <-- CORRECCIÓN: Registros robustos de 32 bits
35 // para prioridades -->
36 #define NVIC_IPR1 *(volatile uint32_t *) (
37 NVIC_BASE + 0x304) // Controla IRQ 4 a 7
38 #define NVIC_IPR2 *(volatile uint32_t *) (
39 NVIC_BASE + 0x308) // Controla IRQ 8 a 11
40 #define SCB_SHPR3 *(volatile uint32_t *) (0
41 xE000ED20) // Controla prioridad del
42 SysTick
43
44 // --- Campos de Bits ---
45 #define RCC_AFIOEN (1 << 0)
46 #define RCC_IOPAEN (1 << 2)
47 #define RCC_IOPCEN (1 << 4)
48 #define GPIOC13 (1UL << 13)
49 #define GPIOA6 (1UL << 6) // LED Botón Baja
50 Prioridad
51 #define GPIOA5 (1UL << 5) // LED Botón Alta
52 Prioridad
53 #define GPIOA4 (1UL << 4) // LED Normalidad en
54 PA4
55
56 // --- Registros del SysTick ---
57 #define SysTick_BASE 0xE000E010
58 #define SysTick_CTRL *(volatile uint32_t *) (
59 SysTick_BASE + 0x00)
60 #define SysTick_LOAD *(volatile uint32_t *) (
61 SysTick_BASE + 0x04)
62 #define SysTick_VAL *(volatile uint32_t *) (
63 SysTick_BASE + 0x08)
64
65 #define SysTick_CTRL_ENABLE (1 << 0)
66 #define SysTick_CTRL_TICKINT (1 << 1)
67 #define SysTick_CTRL_CLKSOURCE (1 << 2)
68
69 volatile uint32_t tick;
70
71 // --- Funciones de Tiempo ---
72 void systick_init_ms(void)
73 {
74     tick = 0;
75     SysTick_CTRL &= ~SysTick_CTRL_CLKSOURCE;
76     SysTick_LOAD = 999;
77     SysTick_VAL = 0;
78     SysTick_CTRL |= SysTick_CTRL_TICKINT |
79 SysTick_CTRL_ENABLE;
80 }
81
82 void delay_ms_bloqueante(uint32_t ms)
83 {
84     uint32_t tiempo_inicio = tick;
85     while ((tick - tiempo_inicio) < ms);
86 }
87
88 // --- Rutinas de Servicio de Interrupción (ISR) ---
89
90 void SysTick_Handler(void)
91 {
92     tick++;
93 }
94
95 // Botón en PA1 (EXTI1) - PRIORIDAD BAJA -> Titila
96 PA6 5 veces
97 void EXTI1_IRQHandler(void)
98 {
99     if (EXTI_PR & (1 << 1))
100     {
101         EXTI_PR = (1 << 1);
102
103         for (int i = 0; i < 10; i++)
104         {
105             GPIOA_ODR ^= GPIOA6;
106             delay_ms_bloqueante(200);
107         }
108
109         GPIOA_ODR &= ~GPIOA6;
110     }
111 }
112
113 // Botón en PA2 (EXTI2) - PRIORIDAD ALTA -> Titila
114 PA5 5 veces
115 void EXTI2_IRQHandler(void)
116 {
117     if (EXTI_PR & (1 << 2))
118     {
119         EXTI_PR = (1 << 2);
120
121         for (int i = 0; i < 10; i++)
122         {
123             GPIOA_ODR ^= GPIOA5;
124             delay_ms_bloqueante(200);
125         }
126
127         GPIOA_ODR &= ~GPIOA5;
128     }
129 }
130
131 // --- Función Principal ---
132 void main(void)
133 {
134     // 1. RCC (Relojes)
135     RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN |
136 RCC_IOPCEN;
137
138     // 2. GPIO
139     // PC13 (LED placa) como salida
140     GPIOC_CRH &= 0xFFFFFFF;
141     GPIOC_CRH |= 0x00200000;
142
143     // Configurar PA6, PA5, PA4 (salidas) y PA2, PA1
144     (entradas)
145     GPIOA_CRL &= 0xF000F00F;
146     GPIOA_CRL |= 0x02220880;
147 }

```

```

124 // Activamos resistencias Pull-up internas en
125 PA1 y PA2
126 GPIOA_ODR |= (1 << 1) | (1 << 2);
127
128 // 3. AFIO (Multiplexor)
129 AFIO_EXTICR1 &= ~(0x00000FF0);
130
131 // 4. EXTI
132 EXTI_IMR |= (1 << 1) | (1 << 2);
133 EXTI_FTSR |= (1 << 1) | (1 << 2);
134
135 // <-- CORRECCIÓN: 5. NVIC PRIORIDADES (32 bits
136 reales) -->
137
138 // a) Garantizar que SysTick tenga la prioridad
139 máxima (0x00)
140 SCB_SHPR3 &= ~(0xFF << 24);
141
142 // b) Limpiar las prioridades previas de EXTI1 (
143 IRQ 7) y EXTI2 (IRQ 8)
144 NVIC_IPR1 &= ~(0xFF << 24); // Limpia bits 24 a
145 31
146 NVIC_IPR2 &= ~(0xFF << 0); // Limpia bits 0 a 7
147
148 // c) Asignar prioridades (recordando que el nú
149 mero menor es la prioridad más alta)
150 // Usamos los bits superiores de cada byte
151 porque STM32 lee los 4 bits más significativos
152 NVIC_IPR2 |= (0x40 << 0); // PA2 (EXTI2) - ALTA
153 Prioridad (0x40)
154 NVIC_IPR1 |= (0x80 << 24); // PA1 (EXTI1) - BAJA
155 Prioridad (0x80)
156
157 // Jerarquía resultante: SysTick (0x00) > PA2 (0
158 x40) > PA1 (0x80)
159
160 // 6. NVIC ENABLE
161 NVIC_ISER0 |= (1 << 7) | (1 << 8);
162
163 systick_init_ms();
164
165 while (1)
166 {
167     // --- Tarea de Normalidad (Background Task)
168     ---
169     GPIOA_ODR ^= GPIOA4;
170     GPIOC_ODR ^= GPIOC13;
171     delay_ms_bloqueante(500);
172 }

```

```

20
21 .weak HardFault_Handler
22 .thumb_set HardFault_Handler, Default_Handler
23
24 .weak WWDG_IRQHandler
25 .thumb_set WWDG_IRQHandler, Default_Handler
26
27 .weak PVD_IRQHandler
28 .thumb_set PVD_IRQHandler, Default_Handler
29
30 .weak TAMPER_IRQHandler
31 .thumb_set TAMPER_IRQHandler, Default_Handler
32
33 .weak RTC_IRQHandler
34 .thumb_set RTC_IRQHandler, Default_Handler
35
36 .weak FLASH_IRQHandler
37 .thumb_set FLASH_IRQHandler, Default_Handler
38
39 .weak RCC_IRQHandler
40 .thumb_set RCC_IRQHandler, Default_Handler
41
42 .weak EXTI0_IRQHandler
43 .thumb_set EXTI0_IRQHandler, Default_Handler
44
45 .weak EXTI1_IRQHandler
46 .thumb_set EXTI1_IRQHandler, Default_Handler
47
48 .weak EXTI2_IRQHandler
49 .thumb_set EXTI2_IRQHandler, Default_Handler
50
51 .weak EXTI3_IRQHandler
52 .thumb_set EXTI3_IRQHandler, Default_Handler
53
54 .weak EXTI4_IRQHandler
55 .thumb_set EXTI4_IRQHandler, Default_Handler
56 // vectores
57 .section .isr_vector,"a",%progbits
58 .word _esstack
59 .word _reset
60 .word NMI_Handler /* 2: NMI (Non-Maskable
61 Interrupt) */
62 .word HardFault_Handler /* 3: HardFault (falla
63 grave) */
64 .word 0 /* 4: MemManage (falla de
65 memoria) */
66 .word 0 /* 5: BusFault (falla de
67 bus) */
68 .word 0 /* 6: UsageFault (falla
69 de uso) */
70 .word 0 /* 7: Reservado */
71 .word 0 /* 8: Reservado */
72 .word 0 /* 9: Reservado */
73 .word 0 /* 10: Reservado */
74 .word 0 /* 11: SVCALL (Llamada al
75 Supervisor) */
76 .word 0 /* 12: Debug Monitor */
77 .word 0 /* 13: Reservado */
78 .word 0 /* 14: PendSV (Interrupció
79 n pendiente del sistema) */
80 .word SysTick_Handler /* 15: El handler del
81 SysTick */
82
83 .word WWDG_IRQHandler
84 .word PVD_IRQHandler
85 .word TAMPER_IRQHandler
86 .word RTC_IRQHandler
87 .word FLASH_IRQHandler
88 .word RCC_IRQHandler
89 .word EXTI0_IRQHandler
90 .word EXTI1_IRQHandler
91 .word EXTI2_IRQHandler
92 .word EXTI3_IRQHandler
93 .word EXTI4_IRQHandler

```

Listing 1. Configuración inicial en main.c

V-B. crt.s(Ejercicio 1)

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
7 text.manejadores
8
9 // función básica para manejar los punteros
10 .thumb_func
11 .weak Default_Handler
12 Default_Handler:
13     b . /* Bucle infinito */
14
15 // definir los vectores ejemplo
16 .weak NMI_Handler
17 .thumb_set NMI_Handler, Default_Handler
18
19 .weak SysTick_Handler
20 .thumb_set SysTick_Handler, Default_Handler

```

```

70 .word 0 /* 12: Debug Monitor */
71 .word 0 /* 13: Reservado */
72 .word 0 /* 14: PendSV (Interrupció
73 n pendiente del sistema) */
74 .word SysTick_Handler /* 15: El handler del
75 SysTick */
76
77 .word WWDG_IRQHandler
78 .word PVD_IRQHandler
79 .word TAMPER_IRQHandler
80 .word RTC_IRQHandler
81 .word FLASH_IRQHandler
82 .word RCC_IRQHandler
83 .word EXTI0_IRQHandler
84 .word EXTI1_IRQHandler
85 .word EXTI2_IRQHandler
86 .word EXTI3_IRQHandler
87 .word EXTI4_IRQHandler

```



```

88 // declaramos la función _reset dentro de la sección
    text.reset
89 .section .text.reset
90 .thumb_func
91 .global _reset
92 _reset:
93     bl main
94     b .

```

V-C. main (Ejercicio 2)

```

1 #include <stdint.h>
2
3 // --- Direcciones Base ---
4 #define RCC_BASE 0x40021000
5 #define GPIOC_BASE 0x40011000
6 #define GPIOA_BASE 0x40010800
7 #define AFIO_BASE 0x40010000
8 #define EXTI_BASE 0x40010400
9 #define NVIC_BASE 0xE000E100
10
11 // --- Registros de Periféricos ---
12 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
    + 0x18)
13 #define GPIOC_CRH *(volatile uint32_t *) (
    GPIOC_BASE + 0x04)
14 #define GPIOA_CRL *(volatile uint32_t *) (
    GPIOA_BASE + 0x00)
15 #define GPIOC_ODR *(volatile uint32_t *) (
    GPIOC_BASE + 0x0C)
16 #define GPIOA_ODR *(volatile uint32_t *) (
    GPIOA_BASE + 0x0C)
17
18 #define AFIO_EXTICR1 *(volatile uint32_t *) (
    AFIO_BASE + 0x08)
19 #define EXTI_IMR *(volatile uint32_t *) (
    EXTI_BASE + 0x00)
20 #define EXTI_FTSR *(volatile uint32_t *) (
    EXTI_BASE + 0x0C)
21 #define EXTI_PR *(volatile uint32_t *) (
    EXTI_BASE + 0x14)
22 #define NVIC_ISER0 *(volatile uint32_t *) (
    NVIC_BASE + 0x00)
23
24 // --- Registros del ADC1 ---
25 #define ADC1_BASE 0x40012400
26 #define ADC1_SR *(volatile uint32_t *) (ADC1_BASE
    + 0x00)
27 #define ADC1_CR2 *(volatile uint32_t *) (ADC1_BASE
    + 0x08)
28 #define ADC1_SMPR1 *(volatile uint32_t *) (ADC1_BASE
    + 0x0C)
29 #define ADC1_SMPR2 *(volatile uint32_t *) (ADC1_BASE
    + 0x10)
30 #define ADC1_SQR3 *(volatile uint32_t *) (ADC1_BASE
    + 0x34)
31 #define ADC1_DR *(volatile uint32_t *) (ADC1_BASE
    + 0x4C)
32
33 // --- Campos de Bits ---
34 #define RCC_AFIOEN (1 << 0)
35 #define RCC_IOPAEN (1 << 2)
36 #define RCC_IOPCEN (1 << 4)
37 #define RCC_ADC1EN (1 << 9)
38
39 #define GPIOC13 (1UL << 13)
40
41
42 #define GPIOA3 (1UL << 3) // Nuevo LED 1
43 #define GPIOA4 (1UL << 4) // Nuevo LED 2
44 #define GPIOA5 (1UL << 5) // Nuevo LED 3
45 #define GPIOA6 (1UL << 6) // Nuevo LED 4
46
47 #define ADC_CR2_ADON (1 << 0)
48 #define ADC_CR2_CAL (1 << 2)

```

```

49 #define ADC_CR2_EXTSEL (7 << 17)
50 #define ADC_CR2_EXTTRIG (1 << 20)
51 #define ADC_CR2_SWSTART (1 << 22)
52 #define ADC_CR2_TSVREFE (1 << 23)
53 #define ADC_SR_EOC (1 << 1)
54
55
56 volatile uint8_t modo_adc = 0;
57
58 // --- ISR del Botón (Pin PA2) ---
59 void EXTI2_IRQHandler(void)
60 {
61     if (EXTI_PR & (1 << 2))
62     {
63         EXTI_PR = (1 << 2);
64         modo_adc = !modo_adc;
65     }
66 }
67
68 void delay_bucle(volatile uint32_t iteraciones) {
69     while (iteraciones > 0) iteraciones--;
70 }
71
72 void main(void)
73 {
74     RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN |
    RCC_IOPCEN | RCC_ADC1EN;
75
76     GPIOC_CRH &= 0xFF0FFFFFF;
77     GPIOC_CRH |= 0x00200000;
78
79     GPIOA_CRL &= 0xF000000F;
80     GPIOA_CRL |= 0x02222800;
81     GPIOA_ODR |= (1 << 2);
82
83     // 3. AFIO y EXTI
84     AFIO_EXTICR1 &= ~(0x00000F00);
85     EXTI_IMR |= (1 << 2);
86     EXTI_FTSR |= (1 << 2);
87     NVIC_ISER0 |= (1 << 8);
88
89     // Inicializar ADC
90     ADC1_CR2 |= ADC_CR2_ADON;
91     delay_bucle(5000);
92     ADC1_CR2 |= ADC_CR2_ADON;
93
94     ADC1_CR2 |= ADC_CR2_CAL;
95     while(ADC1_CR2 & ADC_CR2_CAL);
96
97     ADC1_CR2 |= ADC_CR2_TSVREFE;
98
99     ADC1_CR2 |= ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;
100
101     ADC1_SMPR1 |= (0x7 << 18);
102     ADC1_SMPR2 |= (0x7 << 3);
103
104     uint16_t valor_adc = 0;
105
106     while (1)
107     {
108         GPIOC_ODR ^= GPIOC13;
109
110         if (modo_adc == 0) {
111             ADC1_SQR3 = 16;
112         } else {
113             ADC1_SQR3 = 1;
114         }
115
116         // Iniciar conversión
117         ADC1_CR2 |= ADC_CR2_SWSTART;
118
119         // Esperar EOC (End Of Conversion)
120         while ((ADC1_SR & ADC_SR_EOC) == 0);
121
122         valor_adc = ADC1_DR >> 16;
123
124         // Mostrar valor en pantalla
125         // ... (código para mostrar valor en pantalla) ...
126     }
127 }

```

```

125     valor_adc = ADC1_DR;
126
127     // Apagamos los 4 pines de la barra
128     GPIOA_ODR &= ~(GPIOA6 | GPIOA5 | GPIOA4 |
129     GPIOA3);
130
131     // Encendido progresivo acumulativo
132     if (valor_adc > 819) GPIOA_ODR |= GPIOA3;
133     // Primer nivel
134     if (valor_adc > 1638) GPIOA_ODR |= GPIOA4;
135     // Segundo nivel
136     if (valor_adc > 2457) GPIOA_ODR |= GPIOA5;
137     // Tercer nivel
138     if (valor_adc > 3276) GPIOA_ODR |= GPIOA6;
139     // Cuarto nivel (Máximo)
140
141     delay_bucle(30000);
142 }

```

```

47 .word 0 /* 4: MemManage (falla de
48 memoria) */
49 .word 0 /* 5: BusFault (falla de
50 bus) */
51 .word 0 /* 6: UsageFault (falla
52 de uso) */
53 .word 0 /* 7: Reservado */
54 .word 0 /* 8: Reservado */
55 .word 0 /* 9: Reservado */
56 .word 0 /* 10: Reservado */
57 .word 0 /* 11: SVCcall (Llamada al
58 Supervisor) */
59 .word 0 /* 12: Debug Monitor */
60 .word 0 /* 13: Reservado */
61 .word 0 /* 14: PendSV (Interrupció
62 n pendiente del sistema) */
63 .word SysTick_Handler /* 15: El handler del
64 SysTick */
65
66 /* --- INICIO DE INTERRUPCIONES EXTERNAS (IRQs) ---
67 */
68 /* (Corrección 2: Relleno de las IRQ 0 a 4 para
69 alinear la memoria) */
70 .word 0 /* IRQ0: WWDG */
71 .word 0 /* IRQ1: PVD */
72 .word 0 /* IRQ2: TAMPER */
73 .word 0 /* IRQ3: RTC */
74 .word 0 /* IRQ4: FLASH */
75
76 /* Ahora sí, tus interrupciones caen en su posición
77 correcta por hardware */
78 .word RCC_IRQHandler /* IRQ5: RCC */
79 .word EXTI0_IRQHandler /* IRQ6: EXTI0 */
80 .word EXTI1_IRQHandler /* IRQ7: EXTI1 (PA1) */
81 .word EXTI2_IRQHandler /* IRQ8: EXTI2 (PA2) */
82 .word EXTI3_IRQHandler /* IRQ9: EXTI3 */

```

V-D. crt.s (Ejercicio 2)

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
7 text.manejadores
8
9 // función básica para manejar los punteros
10 .thumb_func
11 .weak Default_Handler
12 Default_Handler:
13     b . /* Bucle infinito */
14
15 // definir los vectores ejemplo
16 .weak NMI_Handler
17 .thumb_set NMI_Handler, Default_Handler
18
19 .weak SysTick_Handler
20 .thumb_set SysTick_Handler, Default_Handler
21
22 .weak HardFault_Handler
23 .thumb_set HardFault_Handler, Default_Handler
24
25 // --- Declaración de tus nuevas interrupciones (
26 Corrección 1) ---
27 .weak RCC_IRQHandler
28 .thumb_set RCC_IRQHandler, Default_Handler
29
30 .weak EXTI0_IRQHandler
31 .thumb_set EXTI0_IRQHandler, Default_Handler
32
33 .weak EXTI1_IRQHandler
34 .thumb_set EXTI1_IRQHandler, Default_Handler
35
36 .weak EXTI2_IRQHandler
37 .thumb_set EXTI2_IRQHandler, Default_Handler
38
39 .weak EXTI3_IRQHandler
40 .thumb_set EXTI3_IRQHandler, Default_Handler
41 //
42
43 -----
44
45 // vectores
46 .section .isr_vector, "a", %progbits
47 .word _esstack
48 .word _reset
49 .word NMI_Handler /* 2: NMI (Non-Maskable
50 Interrupt) */
51 .word HardFault_Handler /* 3: HardFault (falla
52 grave) */

```

```

62 .word 0 /* IRQ0: WWDG */
63 .word 0 /* IRQ1: PVD */
64 .word 0 /* IRQ2: TAMPER */
65 .word 0 /* IRQ3: RTC */
66 .word 0 /* IRQ4: FLASH */
67
68 /* Ahora sí, tus interrupciones caen en su posición
69 correcta por hardware */
70 .word RCC_IRQHandler /* IRQ5: RCC */
71 .word EXTI0_IRQHandler /* IRQ6: EXTI0 */
72 .word EXTI1_IRQHandler /* IRQ7: EXTI1 (PA1) */
73 .word EXTI2_IRQHandler /* IRQ8: EXTI2 (PA2) */
74 .word EXTI3_IRQHandler /* IRQ9: EXTI3 */
75
76 // declaramos la función _reset dentro de la seccion
77 text.reset
78 .section .text.reset
79 .thumb_func
80 .global _reset
81 _reset:
82     bl main
83     b .

```